

MANUAL TÉCNICO

El lenguaje Wison es un lenguaje de configuración diseñado específicamente para esta práctica. Su propósito es permitir al usuario definir gramáticas libres de contexto de forma estructurada, especificando tanto el análisis léxico (terminales) como el análisis sintáctico (no terminales, producciones y símbolo inicial).

Tokens del analizador léxico:

El lexer de wison.json reconoce los siguientes tokens, definidos en orden de prioridad (los tokens más largos y más específicos se definen primero para evitar coincidencias parciales incorrectas):

- **Apertura y cierre de wison:** Estos tokens delimitan el inicio y el fin de todo el código Wison:

Token	Patrón en el lexer	Descripción
WISON_CLOSE	"?"[\t]*"Wison"	Cierre: ?Wison o ? Wison
WISON_OPEN_JOINED	"Wison"[\t]*"\u00BF"	Apertura junta: Wison¿ o Wison ¿
WISON	"Wison"	Palabra clave Wison sola
WISON_OPEN	"\u00BF" (¿)	Símbolo ¿ separado

WISON_CLOSE debe ir antes que WISON porque ambos empiezan con la misma palabra. Si WISON se definiera primero, ?Wison tokenizaría como QUESTION + WISON en lugar del token correcto WISON_CLOSE.

La apertura acepta dos formas porque el símbolo ¿ (U+00BF) puede escribirse pegado o separado de la palabra Wison. El lexer usa el patrón "Wison"[\t]*"¿" para cubrir ambos casos.

- **Delimitadores de bloques:** Los delimitadores de los bloques Lex y Syntax también siguen una regla de orden: el token más largo primero.

Token	Patrón	Uso
SYNTAX_CLOSE	" : } } "	Cierre del bloque Syntax
SYNTAX_OPEN	" { { : "	Apertura del bloque Syntax
LEX_CLOSE	" : } "	Cierre del bloque Lex
LEX_OPEN	" { : "	Apertura del bloque Lex

Si `:}` se definiera antes que `:}}`, el lexer consumiría los primeros dos caracteres `:}` y dejaría un `}` suelto, provocando un error sintáctico. Al definir `:}}` primero, Jison aplica la regla de coincidencia más larga correctamente.

- **Palabras clave:** Todas las palabras clave del lenguaje son case sensitive, exactamente como están escritas:

Token	Lexema	Contexto de uso
NO_TERMINAL_KW	No_Terminal	Declarar un no terminal en Syntax
INITIAL_SIM_KW	Initial_Sim	Declarar el símbolo inicial en Syntax
TERMINAL_KW	Terminal	Declarar un terminal en Lex
SYNTAX	Syntax	Encabezado del bloque sintáctico
LEX	Lex	Encabezado del bloque léxico

- **Nombres de terminales y no terminales:**

TERMINAL_NAME	<code>\\$_[a-zA-Z][a-zA-Z0-9_]*</code>	<code>\$_Numero</code> , <code>\$_FIN</code> , <code>\$_P_Ab</code>
NT_NAME	<code>\%[a-zA-Z][a-zA-Z0-9_]*</code>	<code>%_S</code> , <code>%_Prod_A</code> , <code>%_Expr</code>
NT_NAME	<code>\%[a-zA-Z][a-zA-Z0-9_]*</code>	<code>%S</code> (tolerancia de errores)

La segunda regla para `NT_NAME` (sin guion bajo) es una regla de tolerancia: si el usuario escribe `%S` en lugar de `%_S`, el parser no falla con un error léxico incomprensible. Ambas variantes producen el mismo token `NT_NAME`.

- **Operadores y separadores:**

Token	Lexema	Uso
ARROW	<code><-</code>	Asigna una expresión regular a un terminal
PROD_ARROW	<code><=</code>	Define el cuerpo de una producción
PIPE	<code> </code>	Separador de alternativas en una producción
SEMICOLON	<code>;</code>	Fin de declaración (terminal, NT o producción)
STAR	<code>*</code>	Operador Kleene: cero o más repeticiones
PLUS	<code>+</code>	Cerradura positiva: una o más repeticiones
QUESTION	<code>?</code>	Operador opcional: cero o una vez
LPAREN	<code>(</code>	Agrupación para concatenación de expresiones
RPAREN	<code>)</code>	Cierre de agrupación

- **Expresiones regulares en bloques lex:**

Token	Patrón	Ejemplo
RANGE_ALPHA	"[aA-zZ]"	Terminal \$_Letra <- [aA-zZ] ;
RANGE_DIGIT	"[0-9]"	Terminal \$_Num <- [0-9] ;
LITERAL	'texto'	Terminal \$_FIN <- 'FIN' ;

Los rangos [aA-zZ] y [0-9] son los únicos rangos permitidos en Wison. Se definen como strings literales exactos en el lexer, lo que significa que [A-Z] o [a-z] solos generarían un error sintáctico.

- **Comentarios y espacios:**

Tipo	Patrón	Comportamiento
Comentario de línea	# hasta fin de línea	Ignorado completamente
Comentario de bloque	/** ... */	Ignorado completamente
Espacios y saltos	\s+ (incluye Unicode)	Ignorados completamente

Gramática BNF del lenguaje Wison

A continuación se presenta la gramática completa en notación BNF que describe la estructura del lenguaje Wison, tal como está implementada en wison.json:

```

program
    : wison_open lex_block syntax_block WISON_CLOSE
    ;

wison_open
    : WISON WISON_OPEN          /* Wison ¿ con espacio */
    | WISON_OPEN_JOINED         /* Wison¿ sin espacio */
    ;

lex_block
    : LEX LEX_OPEN terminal_decl_list LEX_CLOSE
    | LEX LEX_OPEN LEX_CLOSE     /* bloque vacio */
    ;

```

terminal_decl

```
: TERMINAL_KW TERMINAL_NAME ARROW regex_expr SEMICOLON
;
```

regex_expr

```
: regex_expr STAR /* e* */
| regex_expr PLUS /* e+ */
| regex_expr QUESTION /* e? */
| LPAREN regex_expr RPAREN regex_expr %prec CONCAT /*
(e1) (e2) */
| LPAREN regex_expr RPAREN /* (e) */
| LITERAL /* 'a' */
| RANGE_ALPHA /* [aA-zZ] */

| RANGE_DIGIT /* [0-9] */
| TERMINAL_NAME /* referencia
*/
;
```

syntax_block

```
: SYNTAX SYNTAX_OPEN nt_decl_list initial_decl
production_list SYNTAX_CLOSE
;
```

nt_decl

```
: NO_TERMINAL_KW NT_NAME SEMICOLON
;
```

initial_decl

```
: INITIAL_SIM_KW NT_NAME SEMICOLON
;
```

production

```
: NT_NAME PROD_ARROW alternative_list SEMICOLON
;
```

```

alternative_list
    : alternative_list PIPE symbol_list    /* A | B */
    | alternative_list PIPE                /* A | ε (épsilon) */
    | symbol_list
    ;

symbol_list
    : symbol_list symbol
    | symbol
    ;

symbol
    : NT_NAME          /* %_S, %_Expr ... */
    | TERMINAL_NAME /* $_A, $_FIN ... */
    ;

```

Resolución del conflicto shift/reduce en regex_expr

Jison resuelve esto por defecto haciendo shift, lo cual puede producir resultados incorrectos en algunos casos. Para hacerlo explícito y controlado, se declararon las siguientes directivas de precedencia:

```

%right QUESTION STAR PLUS    /* operadores unarios, mayor precedencia */
%left  CONCAT                /* concatenación implícita, menor          */

```

Y la regla de concatenación lleva la directiva %prec CONCAT:

```

| LPAREN regex_expr RPAREN regex_expr %prec CONCAT

```

Esto indica al parser que cuando hay ambigüedad entre aplicar un operador unario o continuar una concatenación, el operador unario tiene mayor precedencia. El resultado es que $([0-9]^+)([0-9]^+)$ se interpreta correctamente como la concatenación de dos grupos independientes.

Estructura del AST generado:

Al hacer parse exitoso de un archivo Wison, el parser retorna un objeto JavaScript con la siguiente estructura:

```

{
  terminals: [
    { name: '$_Nombre', pattern: 'regex', patternType: 'tipo' }
  ],
  nonTerminals: [
    { name: '%_Nombre', isInitial: false }
  ],
  initialSymbol: '%_S',
  productions: [
    {
      head: '%_S',
      alternatives: [
        [ {name:'%_A', type:'nonTerminal'}, {name:'$_B',
type:'terminal'} ],
        [] // alternativa vacía = épsilon
      ]
    }
  ],
  errors: []
}

```

Manejo de errores

El parser puede detectar dos tipos de errores: léxicos y sintácticos. La distinción se hace inyectando un `parseError` personalizado antes de cada llamada al parser:

```

parser.yy.parseError = function(msg, hash) {
  captured = {
    type:      hash.token !== undefined ? 'SINTÁCTICO' : 'LÉXICO',
    message:   msg,
    line:      hash.loc.first_line,
    col:       hash.loc.first_column,
    token:     hash.token,           // token inesperado
    expected:  hash.expected         // tokens válidos en ese punto
  };
  throw new Error(msg);
};

```

Sección: Construcción de Gramáticas LL(1)

Un analizador sintáctico LL(1) es un analizador descendente que procesa la entrada de izquierda a derecha y construye la derivación más a la izquierda. El nombre LL(1) indica: L de izquierda a derecha, L de derivación izquierda, y 1 de un símbolo de anticipación (lookahead).

Para que una gramática libre de contexto pueda ser reconocida por un analizador LL(1), debe cumplir tres condiciones estrictas:

- **Sin ambigüedad:** no debe existir más de una derivación izquierda para ninguna cadena.
- **Sin recursión izquierda:** ningún no terminal puede derivar a sí mismo como primer símbolo, ni directa ni indirectamente.
- **Sin la necesidad de factorización:** no debe haber dos alternativas de un mismo NT que comiencen con el mismo símbolo.

El módulo ll-generator.js implementa este proceso completo: valida la gramática, calcula los conjuntos FIRST y FOLLOW, construye la tabla M[A,a] y detecta cualquier colisión que indique que la gramática no es LL(1).

- **Calcular first:** El conjunto FIRST(A) contiene todos los terminales que pueden aparecer al inicio de alguna cadena derivada de A. Si A puede derivar la cadena vacía, entonces ϵ también pertenece a FIRST(A).

Reglas de cálculo (punto fijo iterativo):

- Si X es un terminal: $\text{FIRST}(X) = \{ X \}$
- Si $X \rightarrow \epsilon$ es una producción: $\epsilon \in \text{FIRST}(X)$
- Si $X \rightarrow Y_1 Y_2 \dots Y_n$: agregar $\text{FIRST}(Y_1) - \{\epsilon\}$ a $\text{FIRST}(X)$. Si $\epsilon \in \text{FIRST}(Y_1)$, agregar $\text{FIRST}(Y_2) - \{\epsilon\}$, y así sucesivamente. Si todos los Y_i pueden derivar ϵ , agregar ϵ a $\text{FIRST}(X)$.

Resultado para la gramática de expresiones:

No terminal	FIRST
%_E	{ $\$_id$, $\$_lparen$ }
%_Ep	{ $\$_plus$, ϵ }
%_T	{ $\$_id$, $\$_lparen$ }
%_Tp	{ $\$_star$, ϵ }
%_F	{ $\$_id$, $\$_lparen$ }

- **Calcular follow:** El conjunto FOLLOW(A) contiene todos los terminales que pueden aparecer inmediatamente a la derecha de A en alguna forma sentencial. El símbolo \$ (fin de cadena) siempre pertenece a FOLLOW del símbolo inicial.

Reglas de cálculo:

- **Regla 1:** $\$ \in \text{FOLLOW}(S)$ donde S es el símbolo inicial.
- **Regla 2:** Si $A \rightarrow \alpha B \beta$: $\text{FIRST}(\beta) - \{\epsilon\} \subseteq \text{FOLLOW}(B)$
- **Regla 3:** Si $A \rightarrow \alpha B \beta$ y $\epsilon \in \text{FIRST}(\beta)$: $\text{FOLLOW}(A) \subseteq \text{FOLLOW}(B)$
- **Regla 4:** Si $A \rightarrow \alpha B$: $\text{FOLLOW}(A) \subseteq \text{FOLLOW}(B)$

Resultado para la gramática de expresiones:

No terminal	FOLLOW
%_E	{ \$, \$_rparen }
%_Ep	{ \$, \$_rparen }
%_T	{ \$, \$_plus, \$_rparen }
%_Tp	{ \$, \$_plus, \$_rparen }
%_F	{ \$, \$_plus, \$_rparen, \$_star }

- **Construir tabla $M[A,a]$:** La tabla de análisis LL(1) tiene una fila por cada no terminal y una columna por cada terminal (más \$). Se llena con la siguiente regla:

Para cada producción $A \rightarrow \alpha$:

- Para cada terminal $a \in \text{FIRST}(\alpha) - \{\epsilon\}$: $M[A][a] = A \rightarrow \alpha$
- Si $\epsilon \in \text{FIRST}(\alpha)$: para cada terminal $b \in \text{FOLLOW}(A)$: $M[A][b] = A \rightarrow \alpha$

Si alguna celda recibe más de una producción, hay un conflicto y la gramática no es LL(1).

Tabla M completa para la gramática de expresiones:

NT \ T	\$_id	\$_plus	\$_star	\$_lparen	\$_rparen	\$
%_E	$E \rightarrow T E'$			$E \rightarrow T E'$		
%_Ep		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
%_T	$T \rightarrow F T'$			$T \rightarrow F T'$		
%_Tp		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
%_F	$F \rightarrow id$			$F \rightarrow (E)$		

Las celdas vacías representan un error sintáctico. Ninguna celda tiene más de una entrada, lo que confirma que la gramática es LL(1).

Gramáticas rechazadas por el analizador

- **Gramática con recursión izquierda directa:**

El no terminal $\%_Prod_B$ tiene recursión directa.

$\%_Prod_B \leq \%_Prod_B \%_Prod_C \mid \%_Prod_C ;$

Error detectado: Recursión izquierda directa en " $\%_Prod_B$ " -> " $\%_Prod_B \dots$ "

Solución: transformar a recursión derecha introduciendo $\%_Prod_Bp$:

$\%_Prod_B \leq \%_Prod_C \%_Prod_Bp ;$

$\%_Prod_Bp \leq \%_Prod_C \%_Prod_Bp \mid ;$

- **Gramática con recursión izquierda indirecta:**

La recursión pasa por dos no terminales: $A \rightarrow B \alpha$ y $B \rightarrow A \beta$.

$A \rightarrow B x$

$B \rightarrow A y \mid z$

Error detectado: Recursión izquierda indirecta detectada en "A" y "B". El DFS desde A puede alcanzarse a sí mismo: $A \rightarrow B \rightarrow A$

- **Gramática ambigua:** Dos alternativas de un mismo NT comienzan con el mismo terminal, produciendo una colisión en la tabla M.

$S \rightarrow A a \mid b A$

$A \rightarrow a \mid \epsilon$

FIRST($\%_A$) contiene tanto 'a' como ϵ . Cuando el token es 'a', tanto $A \rightarrow a$ como $A \rightarrow \epsilon$ son candidatos (vía FOLLOW). Esto produce una colisión en $M[A][a]$.

Error detectado: Conflicto en $M["\%_A"]["\$_a"]$: múltiples producciones (vía FOLLOW)

Algoritmo de análisis LL(1)

Tokenización: La cadena de entrada se convierte en tokens usando los patrones regex de los terminales declarados en el bloque Lex. Se aplica longest match: si varios patrones coinciden en la posición actual, se toma el que produce la coincidencia más larga. Los espacios se saltan automáticamente.

Ejemplo: la cadena "id + id" produce los tokens:

Posición	Lexema	Token
0	id	$\$_id$
3	+	$\$_plus$
5	id	$\$_id$

7	\$	\$ (EOF)
---	----	----------

Algoritmo de pila:

La pila inicia con [\$, S] donde S es el símbolo inicial. En cada paso:

- **Cima = \$ y token = \$:** ACEPTAR. La cadena es válida.
- **Cima = \$ y token ≠ \$:** RECHAZAR. Hay tokens sobrantes.
- **Cima es terminal T:** Si T coincide con el token actual, consumir ambos (MATCH). Si no coinciden, error sintáctico.
- **Cima es no terminal A:** Consultar M[A][token]. Si hay entrada, expandir reemplazando A con el cuerpo de la producción (EXPAND). Si la celda está vacía, error sintáctico.

Acción	Pila (tope → derecha)	Entrada restante	Producción
Inicio	\$ %_E	id + id \$	
EXPAND	\$ %_Ep %_T	id + id \$	%_E -> %_T %_Ep
EXPAND	\$ %_Ep %_Tp %_F	id + id \$	%_T -> %_F %_Tp
EXPAND	\$ %_Ep %_Tp \$ _id	id + id \$	%_F -> \$ _id
MATCH	\$ %_Ep %_Tp	+ id \$	consume 'id'
EXPAND	\$ %_Ep	+ id \$	%_Tp -> ε
EXPAND	\$ %_Ep %_T \$ _plus	+ id \$	%_Ep -> + %_T %_Ep
MATCH	\$ %_Ep %_T	id \$	consume '+'
EXPAND	\$ %_Ep %_Tp %_F	id \$	%_T -> %_F %_Tp
EXPAND	\$ %_Ep %_Tp \$ _id	id \$	%_F -> \$ _id
MATCH	\$ %_Ep %_Tp	\$	consume 'id'
EXPAND	\$ %_Ep	\$	%_Tp -> ε
EXPAND	\$	\$	%_Ep -> ε
ACCEPT	\$	\$	Cadena aceptada

